

TaskSetu Production — Comprehensive Bug Report

Generated: 2026-07-16
Scope: Full codebase review (server/ + client/ + root config)
Severity Legend: ● Critical / ● High / ● Medium / ● Low

● Critical Bugs

C1. FileController Broken Permission & Error Checks

File: server/controller/fileController.js
Lines: 221, 231, 283, 287, 297

Five locations where `/* Lines ... omitted */` placeholder comments replaced actual error-response logic. Users without permission proceed to file operations; null files proceed downstream; missing tasks silently fall through.

```
// Line 221 - no-op instead of 403 response
if (!hasPermission) { /* Lines 96-100 omitted */}

// Line 231 - no-op instead of 404 response
if (!task) { /* Lines 105-109 omitted */}

// Line 283 - no-op instead of 403 response
if (!hasPermission) { /* Lines 147-151 omitted */}

// Line 287 - no-op instead of 400 response
if (!req.file) { /* Lines 154-158 omitted */}

// Line 297 - no-op instead of 404 response
if (!task) { /* Lines 163-167 omitted */}
```

C2. Hardcoded JWT Secret Fallback (5 files)

If `process.env.JWT_SECRET` is missing, trivially forgeable fallback keys are used:

File	Line	Fallback
server/auth.js	4	"your-secret-key"
server/services/authService.js	13	"your-secret-key"
server/controller/authController.js	206	"your-secret-key"
server/middleware/roleAuth.js	4	"your-secret-key"
server/mongodb-storage.js	35	"your-jwt-secret-key"

C3. ~~express-rate-limit v8+~~ — ipKeyGenerator Removed

File: server/middleware/rateLimitMiddleware.js:1
Line: 1

`ipKeyGenerator` was removed in `express-rate-limit v7+`. The import will throw at runtime:

```
import rateLimit, { ipKeyGenerator } from 'express-rate-limit';
//                ^^^^^^^^^^^^^^^^^ runtime error
```

Used on lines 40 and 76. Package.json shows "express-rate-limit": "^8.2.1".

C4. Stubbed Organization Settings (No DB Persistence)

File: server/controller/organizationSettingsController.js
Lines: 12–471 (entire file)

All 10 endpoints (`getSettings`, `updateBranding`, `uploadLogo`, `deleteLogo`, `updateNotificationDefaults`, `updateTimezone`, `updateWorkingHours`, `createTemplate`, `getTemplates`, `updateTemplate`, `deleteTemplate`) return HTTP 200 success responses but perform zero database operations. Every mutation is prefixed with `// TODO: — user data is silently discarded`.

C5. Stubbed Role Management (No DB Persistence)

File: server/controller/roleController.js
Lines: 93–271

`createRole`, `updateRole`, `deleteRole` — all return success but never write to any collection. The "validation" on `createRole` (line 109) checks `User` model for `customRoles.name`, which doesn't exist in the schema. Custom roles do not exist.

C6. individual Role Treated as Admin on Frontend

File: client/src/utils/auth.js:119–122
Lines: 119–122

```
const isAdmin =
  user?.activeRole === "org_admin" ||
  rolesArr.includes("individual") || // ← BUG: individual ≠ admin
  rolesArr.includes("org_admin");
```

Individual users gain admin-level UI access, bypassing authorization restrictions.

C7. Broken "Remember Me" — Token Always in localStorage

File: client/src/App.jsx:408-419

When `rememberMe` is `false`, the code first writes `token` to `sessionStorage` then immediately overwrites it in `localStorage`. The `session-only` path is dead code — tokens always persist.

```
storage.setItem("token", result.token); // sessionStorage (wasted)
if (!rememberMe) {
  localStorage.setItem("token", result.token); // immediately overrides
  localStorage.setItem("user", JSON.stringify(result.user));
}
```

C8. React Hook Called from Non-Hook Function

File: client/src/utils/auth.js:128-144

Line: 129

```
export const hasAccess = (requiredRoles = []) => {
  const { user } = useUserRole(); // ← violates Rules of Hooks
```

`hasAccess` is a plain function (no `use` prefix). `useUserRole()` internally calls `useQuery`.

C9. Empty Exported Files

File	Issue
client/src/utils/subtaskAPI.js	Empty file (0 bytes) — imports will fail
client/src/hooks/useNotifications.js	Empty file (0 bytes) — imports will fail

C10. Wrong Auth API Endpoint

Files:

- client/src/components/ProtectedRoute.jsx:22 — `uses /api/auth/me` (should be `/api/auth/verify`)
- client/src/components/admin/AdminLayout.jsx:99 — `uses /api/auth/me` (should be `/api/auth/verify`)

The rest of the codebase `uses /api/auth/verify`. If `/api/auth/me` is not registered on the server, the route guard permanently redirects to login.

C11. Duplicate Notification Preference Key

File: server/services/notificationService.js:1386-1387

```
[TriggerEvent.OVERDUE_ESCALATION]: 'overdue_escalation',
[TriggerEvent.CRITICAL_ESCALATION]: 'overdue_escalation', // ← same key!
```

Both events map to the identical preference key. `CRITICAL_ESCALATION` can never be independently toggled.

C12. organization_id vs organizationId Inconsistency

Scope: Throughout `server/` (45+ occurrences in modals, 100+ occurrences in controllers)

The codebase uses both `organization_id` (snake case, used in Mongoose schemas) and `organizationId` (camel case, used in auth middleware `req.user.organizationId`). Controllers like `reportsController.js` use `$or` as a workaround:

```
const orgId = req.user.organizationId || req.user.organization_id;
```

Queries may silently miss data depending on which field the document uses.

🔴 High-Severity Bugs

H1. JWT Stored in localStorage (XSS-accessible)

File: client/src/utils/auth.js

Lines: 5–6, 15, 19, 25–33, 52

Token and user data stored in `localStorage`, accessible to any XSS vector. No `httpOnly` cookie fallback.

```
export const setAuthToken = (token, user) => {
  localStorage.setItem('token', token);
  localStorage.setItem('user', JSON.stringify(user));
```

H2. Reports — Priority "urgent" vs "Critical" Mismatch

File: server/controller/reportsController.js:256

Line: 256

```
{ name: 'Critical', tasks: tasks.filter(t => t.priority === 'urgent').length }
```

The label is "Critical" but the filter value is 'urgent'. Other code uses 'critical'. The "Critical" priority bucket always shows 0 tasks.

H3. Reports — Redundant isDeleted Filter

File: server/controller/reportsController.js:71-77

```
$or: [
  { isDeleted: { $exists: false } },
  { isDeleted: false },
  { isDeleted: { $ne: true } }
]
```

This covers every document — the `$or` does nothing. Intended filter is `{ $ne: true } or { $in: [false, null] }`.

H4. Notification Role Check Uses Substring Match

File: server/controller/notificationController.js:252

```
if (!['admin', 'super_admin'].some(role => userRole?.includes(role))) {
```

If `userRole` is a string (e.g., `"org_admin"`), `.includes("admin")` returns `true` — granting admin access to unauthorized roles.

H5. import.meta.dirname — Requires Node ≥20.11

Files:

- server/vite.js:59,82
- server/index.ts:370

`import.meta.dirname` is only available in Node.js ≥20.11.0 / ≥21.2.0. May fail with older runtimes.

H6. cron.getTasks() — Requires node-cron v3+

File: server/services/cronJobService.js:1208,1563

```
cron.getTasks().forEach((task) => { task.stop(); });
```

`getTasks()` only exists in `node-cron v3+`. `Package.json` specifies `"node-cron": "^4.1.0"` — safe now but suggests lack of version-aware coding.

H7. Stale Closure in SubtaskContext

File: client/src/contexts/SubtaskContext.jsx:63-82

`refreshCallbackRef.current` is updated but `useMemo` `deps` don't include it (refs can't be deps). Consumers see `refreshCallback: null` even after a valid refresh function is stored.

H8. Mutating React Query Cache

File: client/src/components/RoleSwitcher.jsx:148-152

```
const initialRole = userRoles.sort((a, b) => { // .sort() mutates in place
```

`.sort()` mutates the original array, corrupting React Query's cached data.

🕒 Medium-Severity Bugs

M1. R2 Storage — Env Variables Read at Module Level

File: server/services/r2Storage.js:6-11

```
const r2Enabled = process.env.R2_ENABLED === "true";
const accountId = process.env.R2_ACCOUNT_ID;
// ...
```

If env hasn't been loaded before this module is imported, R2 is silently disabled. Initialization at module level prevents lazy configuration.

M2. Failed Login Audit — Placeholder ObjectId

File: server/utils/auditLogger.js:387

```
entity_id: new mongoose.Types.ObjectId(), // missing required argument
```

In `Mongoose 8+`, `new mongoose.Types.ObjectId()` without arguments should not be used as a meaningful entity ID. This creates a random placeholder that can't be correlated.

M3. Empty AuditLogger File

File: server/utils/auditLogger.js — Contains only a 0-length or 1-line file?

Actual file is 1068 lines, but the `logUserLoginFailed` function (line 387) stores a throwaway `ObjectId` rather than extracting any real identifier from the failed request.

M4. Reports — Role String Inconsistency

File: server/controller/reportsController.js

Uses 'superadmin', 'super_admin', 'Super Admin' interchangeably across lines 57–59, 315–318. Normalization needed.

M5. Three Overlapping License Middleware Systems

Files:

- server/middleware/licenseMiddleware.js
- server/middleware/licenseFeatureMiddleware.js
- server/middleware/newLicenseMiddleware.js

Three separate middleware implementations (checkFeatureAccess, requireFeature, checkFeatureAccess) with different logic paths for the same concept — confusing and divergence-prone.

M6. Dynamic import() in Hot Request Path

File: server/middleware/licenseMiddleware.js:29-36

```
const licenseModule = await import('./newLicenseMiddleware.js');
```

Dynamic import inside every request handler adds latency. Should be imported statically at module level.

M7. mongoose.model () Called Inside Loop

File: server/services/seatManagementService.js:59,125

```
mongoose.model('OrganizationLicensePurchase')
```

Called repeatedly inside loops instead of being cached once at module level.

M8. Object URLs Never Revoked

File: client/src/utils/fileUpload.js:24

```
url: URL.createObjectURL(file) // never followed by URL.revokeObjectURL()
```

Repeated file uploads accumulate unreleased blob URLs, causing memory leaks.

M9. JWT Token Logged to Console

File: client/src/pages/auth/Login.jsx:406

```
console.log("login user : ", result);
```

result contains the JWT token — leaks authentication credentials to browser console.

M10. Full User API Response Logged

File: client/src/utils/auth.js:104-109

```
console.log('Auth verify response user data:', user);
```

Prints complete user object from auth verify API, including server-side fields.

M11. XSS — Brittle Regex in SafeHtml Component

File: client/src/components/common/SafeHtml.jsx:73-88

Regex doesn't handle:

- Single-quoted attributes (href='...')
- Unquoted attributes
- Self-closing <a/> tags

Bypassable if DOMPurify is ever reconfigured.

M12. <script> Tag in JSX (Dead Code)

File: client/src/App.jsx:1183-1185

```
<script>window.location.href = '/settings/user-management';</script>
```

React renders <script> as a text node — the redirect never executes.

● Low-Severity / Informational

#	File	Line	Issue
L1	server/vite.js	37	process.exit(1) inside Vite logger error handler — crashes dev server on warnings
L2	server/controller/dashboardController.js	18	const tomorrow = new Date(todayEnd.getTime() + 1) — adds 1ms not 1 day (works for boundary but misleading)
L3	server/routes.js	~2000+	Single file with 2000+ lines — maintainability concern
L4	client/src/stores/tasksStore.js	21, 194, 380	id: Date.now() + Math.random() produces number ID but consuming components expect string

L5	client/src/hooks/useInactivityLogout.js	20–191	Extensive console.logs about auth timing — session fingerprinting risk
L6	client/src/components/tasks/TaskComments.jsx	49–230	Multiple console.logs dumping comment content, permissions, and user IDs
L7	client/src/services/taskService.js	Multiple	No TypeScript types on API response — fragile to backend changes
L8	client/src/utils/apiClient.js	21–38	Network errors (CORS, DNS) pass through with no user-facing feedback

Summary By Category

Category	Count	Highest Severity
Missing error handling (no-op branches)	5	Critical
Security (JWT, XSS, role bypass)	7	Critical
Data loss (stubbed endpoints)	2	Critical
Auth/Access control	4	Critical
Logic errors	4	High
Performance	3	High
Memory leaks	1	Medium
Console information leakage	4	Medium
Maintainability	3	Low
Total unique bugs found: ~45+		